



PROGRAMMING WITH PYTHON

Lekha A

Computer Applications

PROGRAMMING WITH PYTHON

Data Ingestion and Multiprocessing

Handling API calls

Lekha A

Computer Applications



PROGRAMMING WITH PYTHON

Application Programming Interfaces

- Application Programming Interfaces (APIs) serve as software intermediaries.
- Their role is to enable applications to interact with one another.
- A request must be made to the server to retrieve data before it responds with data.



PROGRAMMING WITH PYTHON

Application Programming Interfaces

- Install requests library.
- APIs fundamentally function as URLs that yield data instead of web pages.
- These endpoints take in requests and send back information in organized formats that applications can easily grasp.



PROGRAMMING WITH PYTHON

Application Programming Interfaces - Get

Import Module

```
import requests
```

Make a GET request to an API endpoint

```
response = requests.get('https://dummyjson.com/posts/1')
```

Check if the request was successful

```
if response.status_code == 200:  
    print('Request successful!')  
    print(response.text)  
else:  
    print(f'Request failed with status code: {response.status_code}')
```

Request successful!

```
{"id":1,"title":"His mother had always taught him","body":"His mother had alwa  
ied to live by this motto. He never looked down on those who were less fortuna  
le he was talking to made him change his mind.","tags":["history","american",
```



PROGRAMMING WITH PYTHON

Application Programming Interfaces - Get

- This code executes a GET request to obtain a single post from a test API.
- The response variable holds all the information that the API has returned.



PROGRAMMING WITH PYTHON

Application Programming Interfaces – Handling JSON Responses

- Most APIs return data in JSON format.

Handling JSON Responses in Python

```
import requests  
response = requests.get('https://dummyjson.com/posts/1')
```

```
if response.status_code == 200:  
    '''Convert JSON response to Python dictionary'''  
    data = response.json()  
    print(f"Post title: {data['title']}")  
    print(f"Post body: {data['body']}")
```

Post title: His mother had always taught him

Post body: His mother had always taught him not to ever think of himself as better than others. He'd tried to live by this motto. He never looked down on those who were less fortunate or who had less money than him. But the stupidity of the group of people he was talking to made him change his mind.

- The `.json()` method automatically converts the response to Python dictionaries and lists.



PROGRAMMING WITH PYTHON

Application Programming Interfaces - Extract Data

- Extract data into a json file.

```
import requests
import json
```

Get data from an API

```
response = requests.get('https://dummyjson.com/posts/2')
data = response.json()
```

Save to a JSON file with nice formatting

```
with open('api_data.json', 'w') as file:
    json.dump(data, file, indent=4)
print('Data saved to api_data.json')
```

Data saved to api_data.json

✓ {} api_data.json

```
root
id 2
title "He was an expert but not in a discipline"
body "He was an expert but not in a discipline that anyone could fully appreciate. He knew how to hold the cone just right so that the soft server ice-cream fell into it at the |
ect cone each and every time. It had taken years to perfect and he could now do it without even putting any thought behind it."
tags [] 3 items
0 "french"
1 "fiction"
2 "english"
reactions
likes 859
dislikes 32
views 4884
userId 91
```




PROGRAMMING WITH PYTHON

Application Programming Interfaces - Extract Data

- Extract multiple data into a json file.

```
import requests
import json
```

```
response = requests.get("https://dummyjson.com/posts")
data = response.json()
posts_list = []
```

```
for post in data["posts"]:
    posts_list.append({"id": post["id"],
                      "title": post["title"],
                      "body": post["body"]
    })
```

```
with open("api_data_1.json", "w") as f:
    json.dump(posts_list, f, indent=4)
print("Data saved to api_data_1.json")
```

```
Data saved to api_data_1.json
```

```
root [] 30 items
  0
  1
    id 2
    title "He was an expert but not in a discipline"
    body "He was an expert but not in a discipline that anyone could fully appreciate. He knew how to hold the cone just right so that the soft server ice-cream fell into it perfect cone each and every time. It had taken years to perfect and he could now do it without even putting any thought behind it."
  2
  3
  4
  5
  6
  7
  8
  9
```



PROGRAMMING WITH PYTHON

Application Programming Interfaces – Extract data

- Extract data into a csv file

```
import requests
import csv
```

Get data from an API

```
response = requests.get("https://dummyjson.com/posts/3")
post = response.json()
```

Save to CSV

```
with open("api_data.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["id", "title", "body"])
    writer.writerow([data["id"], data["title"], data["body"]])
print("Data saved to api_data.csv")
```

Data saved to api_data.csv



api_data.csv

Delimiter: , ▼			
	id	title	body
1	2	pert but not in a discipline	ting any thought behind it.



PROGRAMMING WITH PYTHON

Application Programming Interfaces – Extract data

- Extract multiple data into a csv file

```
import requests
import csv
```

```
response = requests.get("https://dummyjson.com/posts")
data = response.json()
posts = data["posts"]
```

```
with open("api_data_1.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["id", "title", "body"])
    for post in posts:
        writer.writerow([post["id"], post["title"], post["body"]])
print("Data saved to api_data_1.csv")
```

Data saved to api_data_1.csv

 api_data_1.csv

Delimiter: , v			
	id	title	body
1	1	her had always taught him	ade him change his mind.
2	2	pert but not in a discipline	ting any thought behind it.
3	3	rest burned up on the hill.	was nowhere to be seen.
4	4	wanted was a candy bar.	he gun pointed at his face.
5	5	ms were dashed that day.	ams came crashing down.
6	6	ure how he had ended up	o way for him to get down.
7	7	is important to remember.	and as much as you want.
8	8	on and with an open fire.	it's best to slowly rotate it.
9	9	different types of secrets.	etting you killed if you did.
10	10	They rushed out the door.	most important thing of all.
11	11	n't quite yet time to panic.	f into believing there was.
12	12	at things could go wrong.	rything would go that day.
13	13	She wanted rainbow hair.	had done this on purpose.



PROGRAMMING WITH PYTHON

Application Programming Interfaces – Extract data

- Extract data into an XML file

```
import requests
import xml.etree.ElementTree as ET
```

```
response = requests.get("https://dummyjson.com/posts/3")
data = response.json()
```

```
root = ET.Element("post")

id_el = ET.SubElement(root, "id")
id_el.text = str(data["id"])

title_el = ET.SubElement(root, "title")
title_el.text = data["title"]
```

```
body_el = ET.SubElement(root, "body")
body_el.text = data["body"]
```

```
tree = ET.ElementTree(root)
tree.write("api_data.xml", encoding="utf-8", xml_declaration=True)
print("Data saved to api_data.xml")
```

Data saved to api_data.xml

  api_data.xml

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
<?xml version="1.0" encoding="utf-8" ?>
<post>
  <id>3</id>
  <title>Dave watched as the forest burned up on the hill.</title>
  <body>Dave watched as the forest burned up on the hill, only a few miles from her house. The car had beer
mental list of the most important papers and documents that they couldn't leave behind. He scolded himsel
needed. He continued to wait for Marta to appear with the pets, but she still was nowhere to be seen.</bc
</post>
```



PROGRAMMING WITH PYTHON

Application Programming Interfaces – Extract data

- Extract multiple data into an XML file

```
import requests
import xml.etree.ElementTree as ET
```

```
response = requests.get("https://dummyjson.com/posts")
data = response.json()
```

```
root = ET.Element("posts")

for item in data["posts"]:
    post_el = ET.SubElement(root, "post")
    id_el = ET.SubElement(post_el, "id")
    id_el.text = str(item["id"])
    title_el = ET.SubElement(post_el, "title")
    title_el.text = item["title"]
    body_el = ET.SubElement(post_el, "body")
    body_el.text = item["body"]
    tree = ET.ElementTree(root)
tree.write("api_data_1.xml", encoding="utf-8", xml_declaration=True)

print("Data saved to api_data_1.xml")
```

Data saved to api_data_1.xml

  api_data_1.xml

```
▼ <posts>
  ► <post>
    ...
    </post>
  ► <post>
    ...
    </post>
  ▼ <post>
    <id>3</id>
    <title>Dave watched as the forest burned up on the hill.</title>
    <body>Dave watched as the forest burned up on the hill, only a few miles from
    mental list of the most important papers and documents that they couldn't leave
    was needed. He continued to wait for Marta to appear with the pets, but she st
    </post>
  ▼ <post>
    <id>4</id>
    <title>All he wanted was a candy bar.</title>
    <body>All he wanted was a candy bar. It didn't seem like a difficult request t
    with the gun pointed at his face.</body>
  </post>
```



PROGRAMMING WITH PYTHON

Application Programming Interfaces - Handling API Errors in Python

- Using Try-Except for Failed Requests

```
import requests
try:
    response = requests.get('https://api.postalpincode.in/pincode/560040', timeout=5)
    response.raise_for_status() # Raises an exception for 4XX/5XX responses
    data = response.json()
    print(f"Retrieved {len(data)} records")
except requests.exceptions.HTTPError as e:
    print(f"HTTP error: {e}")
except requests.exceptions.ConnectionError:
    print("Connection error: Check your internet connection")
except requests.exceptions.Timeout:
    print("Timeout error: The server took too long to respond")
except requests.exceptions.RequestException as e:
    print(f"Error making request: {e}")
except ValueError:
    print("Error parsing JSON response")
```

Connection error: Check your internet connection



PROGRAMMING WITH PYTHON

Application Programming Interfaces - Managing API Rate Limits

- Some APIs limit how many requests can be made per minute or day.

```
import requests
import time
```

```
max_retries = 3
retry_delay = 2 # seconds
for attempt in range(max_retries):
    response = requests.get('https://api.example.com/data')
    # If we hit a rate limit (usually status code 429)
    print(response.status_code)
    if response.status_code == 429:
        print(f"Rate limited. Waiting {retry_delay} seconds...")
        time.sleep(retry_delay)
        retry_delay *= 2 # Exponential backoff to prevent rate limiting
    else:
        break # Success or different error displays here
```




PROGRAMMING WITH PYTHON

Recap

- API's
- Get
- Handling JSON Responses
- Extract Data
- Handling API Errors



THANK YOU

Lekha A

Computer Applications